

SpringBoot 笔记

SpringBoot 工程框架参考

```
Situ_project_resource
├─.idea
├─Situ_project_resource
├─src
│   └─main
│       └─java
│           └─org
│               └─example
│                   ├──config
│                   ├──controller
│                   ├──dto
│                   ├──entity
│                   ├──mapper
│                   ├──service
│                   │   └─Impl
│                   └─utils
│               └─resources
│                   └─static
│                       ├──js
│                       ├──layui
│                       │   ├──css
│                       │   │   └─modules
│                       │   ├──laydate
│                       │   │   └─default
│                       │   └─layer
│                       │       └─default
│                       ├──font
│                       ├──lay
│                       │   └─modules
│                       ├──lib
│                       ├──modules
│                       └─tpl
│                           └─system
│                       ├──User
│                       ├──UserType
│                       └─User_project
│   └─test
│       └─java
└─target
    ├──classes
    │   ├──org
    │   │   └─example
    │   │       ├──config
    │   │       ├──controller
    │   │       ├──dto
    │   │       ├──entity
    │   │       ├──mapper
    │   │       └─service
```

```

| | | ↳ Impl
| | | ↳ utils
| | ↳ static
| |   ↳ js
| |   ↳ layui
| |   | ↳ css
| |   | | ↳ modules
| |   | |   ↳ laydate
| |   | |   | ↳ default
| |   | |   ↳ layer
| |   | |   ↳ default
| |   | ↳ font
| |   | ↳ lay
| |   | | ↳ modules
| |   | ↳ lib
| |   | ↳ modules
| |   | ↳ tpl
| |   | ↳ system
| |   ↳ User
| |   ↳ UserType
| |   ↳ User_project
↳ generated-sources
| ↳ annotations
↳ generated-test-sources
| ↳ test-annotations
↳ test-classes

```

IDEA 集成开发环境

maven 构建框架

1. 创建项目 (maven)
2. 在 `settings.xml` 内设置maven路径

```
<localRepository>D:\MavenRepository</localRepository>
```

- (1)修改settings文件
- (2)修改idea配置
3. 设置IDEA提示大小写
4. lombok 安装
5. 确保spring boot完整
6. 数据库设置 安装 设置
数据库启动端口默认为3306
数据库格式为utf8mb4, InnoDB
默认密码为root
用navicat创建数据库
7. 项目执行
spring boot
spring
spring mvc

mybatis
lombok
mybatis puls
jackson

环境配置

1. pom.xml

1. 父项目中导入的dependency子项目会全部继承
2. 运行工程前，确保IDE运行时选择的jdk版本与工程所使用的jdk版本是一致的，这样才能最大程度使工程成功运行

工程构建流程 (实训期间)

1. 创建java-maven工程，选择jdk版本并贯彻工程
2. 配置工程Project(父工程)总pom表
3. 创建子工程模块包subProject，配置其pom表
4. 在subProject.src.main.java中创建放置controller, mapper, pojo等SpringBoot框架模块的主包，一般会默认为org.example，也可以自命名为 <域名>.<工程名/有意义的名>

至此，工程Project内文件树结构如下：

```
Project
| pom.xml
| tree.txt
└─subProject
    | pom.xml
    |
    └─src
        | └─main
            | | └─java
                | | | └─org
                    | | |     └─example
                        | | |         DemoApplication.java
                        | | |         └─controller
                        | | |         └─mapper
                        | | |         └─pojo
                        | | |             User.java
                        | | └─resources
                            | |     application.yml
                            | |     └─public
                                | |         index.html
                                | └─static
                            └─test
                                └─java
└─target
```

```

└─classes
  │   application.yml
  │   |
  │   └─org
  │       └─example
  │           DemoApplication.class
  │       |
  │       └─public
  │           index.html
  |
  └─generated-sources
      └─annotations

```

5. 编辑pojo包

1. 根据创建的数据库中表的名称，创建表的pojo对象（自主创建）

如：现有数据库如下

1. 数据库database
 1. 表user
 1. uid: int
 2. uname: varchar
 3. upass: varchar
 4. regtime: date
 5. phone: varchar

则创建的User.class中代码如下

```

@TableName(value = "user")
public class User implements Serializable {
    @TableId(value = "uid")
    private Integer uid;
    private String uname;
    private String upass;
    private String phone;
    private Date regtime;

    //Accessors and Mutators are also needed
}

```

主码有且仅有一个属性可以对应

6. 在controller包中创建UserController.class

使用 `@Controller` 将UserController标注为控制器类

```

@Controller
public class UserController {

    //SpringBoot框架中，pojo对象的使用均为注解自动装配，无需使用new关键字创建对象再使用

    //装配接口的代理对象
    @Resource
    private UserMapper userMapper;

    //设置该方法的http请求路径
}

```

```

@RequestMapping("/showUser")
public String showUser(){
    List<User> users = userMapper.selectList(null);
    System.out.println(users.size());
    return "show.html";
}
}

```

7. 对应地，在mapper包中创建UserMapper.class(Interface)

```

@Mapper
public interface UserMapper extends BaseMapper<User> {
    //继承BaseMapper后，该类具备自动生成User的Mapper属性的功能 -> SpringBoot框架下自动装配功能的应用
}

```

8.

通过控制器在网页中从数据库内查询数据

1. 创建表并先添加数据，数据库配置应对应application.properties文件

```

spring.datasource.url=jdbc:mysql://localhost:3308/situ_user?
serverTimezone=Asia/Shanghai&zeroDateTimeBehavior=CONVERT_TO_NULL&autoReconnect=true&useSSL=false&failOverReadOnly=false&allowPublicKeyRetrieval=true

```

3308 为mysql启动端口

situ_user 为创建的数据库名称

.properties文件中为静态配置，在运行时无法修改

2. 创建对应pojo对象 @Data getters setters

即创建实体类，如下：

```

//wriiten in org.example.entity/User.java
package org.example.entity;
import lombok.Data;
@Data
public class User {
    private Integer id; //Integer为引用类型
    private String name;
}

```

3. 创建mapper接口 @Repository 以及 @select 方法

4. 在主启动类加 @MapperScan("org.example.mapper")

5. 在控制器 获取mapper对象

```
@Autowired
User mapper mapper;
```

6. 使用mapper查询，返回值

```
@RequestMapping("select_database")
public List<User> select_database(User user, HttpServletRequest request,
HttpSession session){ //会返回json字符串
    return mapper.select();
}
```

创建一个具备前端后端的数据库查询表流程

1. 创建表
2. 创建pojo对象
3. 创建mapper
4. 创建完整的service包
 1. 外部interface
 2. 内部Impl
5. 创建controller控制器
6. 若列表头改变，则pojo对象和前端html中与列相关的操作都需要做改变

杂项

- 项目开发的三大特性
 - 属性：可被区别的特征 按需求
 - 方法(行为)：具体事物的行为 -> 实质是具体事物属性的改变或基础方法的调用(改变自身的属性) / 由外向内接收信号
 - 事件：由事物内部向外发出信号
- Server端
 - 接收客户端的请求 -> 根据请求进行操作 -> 返回操作结果 -> 返回请求
- 工程框架
 1. spring
 2. spring MVC
 - 在主包中创建controller包 -> 专门用来接待接受请求的角色
 - 对于MVC:
 1. C -> controller: 专门用来接待网页请求，分派业务，回应请求（路由地址接收口）
 1. 内置DispatcherServlet,请求路由器，根据url地址，动态调用容器内的控制器方法
 2. Map集合 -> key:路由地址 value:方法对象
 2. V -> view: 视图，专门用来放置显示数据的页面，不能包含业务
 3. M -> model: 业务模板，实际进行相关业务操作的**控制器中禁止任何的方法重载!!!** -> 对于系统参数来说会分不清调用方法
 3. mybatis
 - 对象到方法的映射

4. lombok

5. mybatis puls

6. jackson

将java语句转换为前端相关的语句

- 主包：Main方法类所在的包/文件夹

- lombok

放置在仅有属性方法的类中，会自动为pojo对象自动添加Accessor和Mutator(get和set方法)，非常省事

会在编译时自动添加

```
import lombok.Data;
@Data
public class User {
    private Integer id;
    private String name;
}
```

- 工程模式

1. 生命周期模式

2. 过去-现在-未来 模式

后端

- why启动即可通过映射调用相关方法：

加入容器方式：

1. 自动扫描主包及内部子包
2. 在主类中使用@ComponentScan("<包名>")自助扫描包
3. 配置清单文件.xml，指定类实例，加入容器 -> 很少使用!!!

- 注解(也用到了反射机制)

1. @RequestMapping //映射

2. @RestController //声明为控制器并加入容器、

3. @Repository //声明为仓库(数据库)后端文件并加入容器

4. 数据库相关方法

1. @Select

2. @Delete

3. @Update

5. @MapperScan("org.example.mapper")

扫描mapper包，将mapper接口通过cglib代理模式，自动生成实现类(加入内部数据库链接)

6. @Service //声明为业务(服务)

7. @Configuration //容器初始化后，配置对象初始化

8. @Bean //一般跟@Configuration配套使用，声明当前方法的返回值并加入容器，添加动态配置对象

- 方法参数

1. mapper 放入数据列信息的方式:

1. `#{}` 必须放到有效位置才可生效, 不能作为sql替代使用, 仿sql注入???
2. `${}` 直接替代sql字符串, 非安全
3. `@Param("<使用时参数名>")` 用于传入多参数, 在数据库中查找多参数均匹配的数据
三种从数据库中查找单项数据的方式:

```
//written in User_mapper.java(Interface)

@Select("select * from user where ${id}")
public User select_single1(@Param("id") Integer id);

@Select("select * from user where id = ${id}, name = ${name}")
public User select_single2(@Param("id") Integer id, @Param("name")
String name);

//该方法使用时不用定义pojo对象, 但使用时更加灵活, 并且该方法可以返回所有符合条件的数据, 可以为多条数据
@Select("select * from user where id = ${id}, name = ${name}")
public List<HashMap<String, Object>> select_single3(@Param("id")
Integer id, @Param("name") String name);
```

4. 实例

```
//written in User_controller.java

@RequestMapping("select_single1")
public User_test select_single1(User_test user, HttpServletRequest
request, HttpSession session){
    return mapper.select_single1(user.getId());
}

@RequestMapping("select_single2")
public List<User_test> select_single2(User_test user,
HttpServletRequest request, HttpSession session){
    return mapper.select_single2(user.getId(), user.getName());
}
```

```
//written in User.mapper.java

@Select("select * from user_test where id = #{id}")
public User_test select_single1(Integer id);

@Select("select * from user_test where id = #{id} or name = #{name}")
public List<User_test> select_single2(@Param("id") Integer id,
@Param("name") String name);
```

2. 基本类型参数 -> 默认值为0, 不能为null -> 必录参数!!!
3. final参数类型 -> 非必录参数, 例如:


```
@RequestMapping("select")
public String select(Integer cnt){
    return "Hello\t + " + cnt;
}
```

4. 引用类型参数 -> 根据参数名称, 将参数值调用对象内相同属性的setter方法

5. 系统参数: 根据参数类型, 由DispatcherServlet自动提供

如: HttpServletRequest request, HttpSession session

- 返回值

1. 数据本身, 如基本类型参数/final(常量)参数类型

2. 引用类型参数 -> 自动调用jackson框架, 转换对象为json字符串

3. void

- 非注入型扩展——mybatis plus

使链接数据库的mapper类继承BaseMapper<数据库数据类型>, 如

```
public interface User_mapper extends BaseMapper<User> {
    ...
}
```

使用该方法可以快速调用BaseMapper类中封装好的各种数据库相关方法, 对于简单操作可以直接调用, 复杂操作则可以选择在mapper.java中进行重载

例如:

```
@RequestMapping("select_id")
public List<User_test> select_id(User_test user, HttpServletRequest request,
HttpSession session){
    return mapper.selectList(new Querywrapper<User_test>().eq("id",
user.getId() ) );
}
```

- QueryWrapper -> Entity对象封装操作类, 需要指明打包返回的查询对象的实体类型

- POJO对象 -> 即简单java对象/类

- json字符串

{ }对象

[]数组

例如:

```
{"id":null, "name":123}
```

实现代码如下:

```
@RequestMapping("select_user")
public User select(User user, HttpServletRequest request, HttpSession
session){ //会返回json字符串
    return user;
}
```

- tip:若想同时输入id和name两个属性值, 可以使用如下格式:

```
http://127.0.0.1/User/select_user?id=1&name=111
```

1. key值: 必须加双引号
2. value值: 数字和boolean可以无双引号
其内部可以无限嵌套, 如对象内部嵌套数组, 数组内又嵌套对象 -> 可以用json在线解析来分析json文件
- 自动注入
 1. bytype -> @Autowired 根据类型自动从容器中获取对象
 2. byname -> @Resource("<名称>") 根据别名名称自动从容器中获取对象
 3. by构造器
- 服务应用层
在主包下添加service包, 将有关数据库的具体实现代码放到service中, 以集中封装, 在控制器中先声明service对象后只需调用即可
- 数据传输对象包dto
ResultData
ResultInfo
SearchInfo -> 接收前台接收的查询信息
- 反射 -> java核心特性
 1. 类描述类 Class cls = User.class;
创建class类对象
 2. 字段描述类 Field f = cls.getDeclaredField("name");
 3. 方法描述类
- 动态配置 @Configuration
- 服务器变量域
 1. page
页面内定义的变量, 只在页面内生效, 可跨绘画
只要控制器未刷新, 定义在该页面的变量仍可被访问
 2. request
请求转发时
服务器更换地址的两种方式:
 1. 请求转发
 2. 地址转发
请求会区分用户, 不同用户在请求内创建的变量并不互通通过get()或post()发送的变量名, 和controller中@RequestMapping()接收的变量名需要保持一致
 3. session 会话
 1. session会话id -> 浏览器每次创建链接时会自动生成不重复的会话id, 并分配专门的隔离会话存储空间, 该隔离区域无法长期保留, 只会存在一段时间
会话有效时间可自行设置, 可体现为每次登录用户后, 在一定时间内刷新用户主页不会被拦截至登录页
且会话有效时间为相对时间, 从最后一次成功提交登录请求后开始计时
只要会话激活后超过了有效时间, 会自动清除会话存储空间内容, 但对应的会话id并不会清除
 2. 会话存储空间

3. 会话跟踪 -> 若误关闭浏览器，则可以根据会话id跟踪会话找回内容
但因为安全问题较少使用，而大多采用talking方式
4. application 可理解为服务器公用变量
与服务器关联，早期时用于放缓存数据

业务实现流程

假设现有数据库sdut_hospital，内含表checkitem，需要实现网页中表格分页展示功能

1. 在前端中设计好视觉组件标签，由标签内置函数转到后端 `<script>` 块内进行请求函数的编写，发送异步请求post()或get()，一般由地址映射和传递参数构成，以及 `then{前端展示数据处理}-catch{异常处理}`，catch为非必要属性

```
<el-button @click="findPage()" class="dalfBut">查询</el-button>
```

```
findPage() {  
    //编写分页查询js代码  
    //1:提供分页数据  
    var params={  
        currentPage: this.pagination.currentPage,  
        pageSize: this.pagination.pageSize,  
        queryString: this.pagination.queryString  
    }  
    //2:发送异步请求  
    //请求传值方式：1. url栏地址直接映射，如 /<methodMapping>?<variable>=<value>  
    //2. json格式对象传值  
    axios.post("/checkitem/findPageInfo",params).then((res)=>{  
        this.pagination.total = res.data.total;  
        this.dataList = res.data.rows;  
    });  
}
```

2. 在映射对象对应的controller中创建请求映射的方法findPageInfo()

tips:

1. 直接从前端html文件中复制映射的方法名，在controller中直接使用
2. 引入checkItemService服务对象后，在findPageInfo()方法中，直接写
checkItemService.findPageInfo(queryPageBean)，不管代码是否爆红，后面再补方法的实现，findPageInfo直接粘贴即可

```
package com.sdut.controller;  
  
import com.sdut.pojo.Checkitem;  
import com.sdut.service.CheckItemService;  
import com.sdut.util.PageResult;  
import com.sdut.util.QueryPageBean;  
import com.sdut.util.Result;  
import org.springframework.web.bind.annotation.RequestBody;  
import org.springframework.web.bind.annotation.RequestMapping;  
  
import org.springframework.web.bind.annotation.RestController;  
  
import javax.annotation.Resource;
```

```

@RestController
@RequestMapping("/checkitem")
public class CheckitemController {
    @Resource
    private CheckItemService checkItemService;

    //展示检查项
    @RequestMapping("/findPageInfo")
    //QueryPageBean是提前在util包中封装好的页面查询对象组件模型
    public PageResult findPageInfo(@RequestBody QueryPageBean queryPageBean){
        PageResult pageResult = checkItemService.findPageInfo(queryPageBean);
        return pageResult;
    }
}

```

QueryPageBean对象数据一览

```

package com.sdut.util;

import java.io.Serializable;

/**
 * 封装查询条件 vo
 */
public class QueryPageBean implements Serializable{
    private Integer currentPage;//页码
    private Integer pageSize;//每页记录数
    private String queryString;//查询条件

    public Integer getCurrentPage() {
        return currentPage;
    }

    public void setCurrentPage(Integer currentPage) {
        this.currentPage = currentPage;
    }

    public Integer getPageSize() {
        return pageSize;
    }

    public void setPageSize(Integer pageSize) {
        this.pageSize = pageSize;
    }

    public String getQueryString() {
        return queryString;
    }

    public void setQueryString(String queryString) {
        this.queryString = queryString;
    }
}

```

3. 在工程中的com.sdut.service包中补齐服务接口类与服务的实现类

该步骤的流程：

1. 先构建好com.sdut.service包和com.sdut.service.impl包
2. service包中创建服务接口类CheckItemService， service.impl包中创建服务实现类CheckItemServiceImpl
3. 从CheckitemController的爆红未命名方法findPageInfo出发，由IDEA自带的debug功能跳转到接口类CheckItemService中，并自动创建对应抽象方法findPageInfo()
4. 再从服务接口类CheckItemService的爆红问题(服务实现类未实现抽象方法)出发，跳转到服务实现类CheckItemServiceImpl中，并自动实现抽象方法findPageInfo()
5. 最后在服务实现类CheckItemServiceImpl中完成具体业务代码的编写

```
package com.sdut.service;

import com.sdut.pojo.Checkitem;
import com.sdut.util.PageResult;
import com.sdut.util.QueryPageBean;
import com.sdut.util.Result;

public interface CheckItemService {
    PageResult findPageInfo(QueryPageBean queryPageBean);
}
```

```
package com.sdut.service.impl;

import ...;

//添加@Service目的是为了将该服务加到SpringBoot框架中进行管理
@Service
public class CheckItemServiceImpl implements CheckItemService {

    @Resource
    private CheckitemMapper checkitemMapper;    //数据库中的表checkitem与后端进行交互的中间对象

    @Override
    public PageResult findPageInfo(QueryPageBean queryPageBean) {
        //判断是否携带模糊查询的条件    条件查询->lambdaQueryWrapper
        String queryString = queryPageBean.getQueryString();
        LambdaQueryWrapper<Checkitem> lambdaQueryWrapper = null;
        if(queryString != null && queryString.length() > 0){
            lambdaQueryWrapper = new LambdaQueryWrapper<>();
            lambdaQueryWrapper.like(Checkitem::getCode, queryString);
            lambdaQueryWrapper.or();
            lambdaQueryWrapper.like(Checkitem::getName, queryString);
        }

        //当前页 每页显示的条数
        Page<Checkitem> page = new Page<>(queryPageBean.getCurrentPage(),
        queryPageBean.getPageSize());
        checkitemMapper.selectPage(page, lambdaQueryWrapper);
        return new PageResult(page.getTotal(), page.getRecords());
    }
}
```

逆向工程

1. Tools: lombok (plugin of IDEA)

2. 作用:

1. 可以视作代码生成器，设置好工程yml文件属性、链接的数据库以及其中表名后，可以直接通过包中封装好的函数进行自动生成pojo, controller等类包的生成
2. 简化工程基础配置操作，提高开发效率

3. Tips:

1. 只会生成类，不会生成类中的业务方法
2. pom中需要导入lombok依赖

4. 示例代码

tips: 生成器代码需要mybatis-plus-generator 3.3.0支持，pom表依赖如下

```
<dependency>
  <groupId>com.baomidou</groupId>
  <artifactId>mybatis-plus-generator</artifactId>
  <version>3.3.0</version>
</dependency>
```

```
package com.sdut;

import com.baomidou.mybatisplus.annotation.DbType;
import com.baomidou.mybatisplus.generator.AutoGenerator;
import com.baomidou.mybatisplus.generator.config.*;
import com.baomidou.mybatisplus.generator.config.rules.NamingStrategy;

public class LombokGenerator {
    public static void main(String[] args) {
        AutoGenerator autoGenerator = new AutoGenerator(); //自动生成器
        GlobalConfig globalConfig = new GlobalConfig(); //全局配置对象

        //全局配置参数
        globalConfig.setOpen(true);
        globalConfig.setAuthor("QHaooolG");
        //代码生成路径

        globalConfig.setOutputDir("F:\\CodingBox\\IntelliJ_IDEA_workspace\\Database_PracticalTraining\\Resources");
        globalConfig.setFileOverride(true); //路径覆盖设置
        autoGenerator.setGlobalConfig(globalConfig);

        // 数据源配置(根据自己数据的连接条件)
        DataSourceConfig dsc = new DataSourceConfig();
        dsc.setUrl("jdbc:mysql://localhost:3307/sdut_hospital");
        dsc.setDriverName("com.mysql.cj.jdbc.Driver");
        dsc.setUsername("root");
        dsc.setPassword("root");
        dsc.setDbType(DbType.MYSQL);
```

```

autoGenerator.setDataSource(dsc);

// 包配置
PackageConfig pc = new PackageConfig();
pc.setParent("com.sdut");
pc.setController("controller");
pc.setEntity("pojo");
pc.setMapper("mapper");
autoGenerator.setPackageInfo(pc);

TemplateConfig templateConfig = new TemplateConfig(); //临时配置
//设置临时配置中的服务接口ServiceImpl以及服务Service不生成
templateConfig.setServiceImpl(null);
templateConfig.setService(null);
//将上述设置加入自动生成器中
autoGenerator.setTemplate(templateConfig);

StrategyConfig strategyConfig = new StrategyConfig();
//数据库表映射到实体的命名策略
strategyConfig.setNaming(NamingStrategy.underline_to_camel);
//数据库表字段映射到实体的命名策略
strategyConfig.setColumnNaming(NamingStrategy.underline_to_camel);
strategyConfig.setEntityLombokModel(true); // lombok 模型
strategyConfig.setRestControllerStyle(true); //restful api 风格控制器

String tableNames=
"user,role,persion,userrole,rolepersion,ordersetting,checkitem,checkgroup,set
meal,orders,paylog,member,checkgroupcheckitem";
strategyConfig.setInclude(tableNames.split(","));
autoGenerator.setStrategy(strategyConfig);

//执行
autoGenerator.execute();
}
}

```

5. 直接运行main函数即可

数据库

创建表头属性时，属性名首字母必须小写，严禁出现大写字母! -> 在java中小写字母才能通过lombok自动创建get/set方法

- 数据库中的配置要与 .properties 或 .yaml 配置文件中相同
- 若数据库中两个表之间存在中间表或视图，则必须先删除中间表，再删除表
若顺序倒置，则会导致删除表时会因表中字段被中间表使用而无法删除
- 若想要在后端代码中对数据库中的表进行操作，则必须先引入对应表的mapper对象

```

@Resource
private SetmealMapper setmealMapper;

```

- 数据库中表的删除形式：

1. 物理删除

直接删除主表以及中间表中指定的所有信息

2. 逻辑删除

只删除中间表中指定信息的记录，但保留主表中的记录

- 若业务中涉及到两个或以上的表的增删改查等操作，则建议在服务接口的实现方法中加入事务管理 @Transactional，例如

```
@Override
@Transactional
public Result deleteInfoById(Integer id, String img) {
    //删除套餐对应的中间表记录
    LambdaQueryWrapper<Setmealcheckgroup> wrapper = new
LambdaQueryWrapper<>();
    wrapper.eq(Setmealcheckgroup::getSetmealId, id);
    boolean flag = false;
    flag = setmealcheckgroupMapper.delete(wrapper) > 0;

    //删除套餐
    flag = setmealMapper.deleteById(id) > 0;

    //删除磁盘中的图片
    String path =
"F:\\CodingBox\\IntelliJ_IDEA_workspace\\Database_PracticalTraining\\Resource
s\\hospital_setmeal_pics\\" + img;
    File file = new File(path);
    file.delete();

    if(flag)
        return new Result(flag, "删除套餐成功");
    else
        return new Result(flag, "删除套餐失败");
}
```

前端

主要使用layui

- Note

1. 前端中任一标签都可以视作一个对象
2. 前端html文件中若想使用js文件中的对象，则需要先在 <head></head> 块中引入（效果与头文件相似）。例如：

```
<head>
  <meta charset="UTF-8">
  <title>Title</title>
  <script type="text/javascript" src="js/jquery.min.js"></script>
  <script type="text/javascript" src="js/axios-0.18.0.js"></script>
  <script type="text/javascript" src="js/vue.js"></script>
</head>
```


3. Script

1. `<script>` 块内使用`var`定义的变量，可以在不同 `<script>` 块间使用；而使用`const`定义的变量则只能在定义块内使用
2. 标签`id`可视为后端中的变量来使用。同样地，使用其他标签类型定义的标签值也可以作为变量使用，如

```
<body>
<div id="app">
  <!-- 这里v-text v-if都可以视作标签类型-->
  <span v-text="info">text1</span>
  <span v-text="msg">text2</span>

  <span v-if="f1">text1</span>
  <span v-show="f2">text2</span>
</div>
</body>

<script type="text/javascript">
  var obj = new Vue({
    //el(element)属性先填入前端标签id，指定前端对象进行Vue操作 -> 筛选器"#<id_name>"
    el: "#app",
    data: {
      //会将对应v-text中的内容进行更新
      info: 'admin',
      msg: '消息',
      f1: true,
      f2: false
    }
  });
</script>
```

4. Vue

1. v-text / v-show

vue中用于显示文本或展示其他内容的标签类型

2. v-if

可控制标签可视性

true -> 显示

false -> 不显示

5. axios

1. 同步 -> 阻塞模式

指一个进程在执行某个请求的时候，若该请求需要一段时间才能返回信息，那么这个进程将会一直等待下去，直到收到返回信息才继续执行下去

2. 异步 -> 非阻塞模式

异步是指进程不需要一直等下去，而是继续执行下面的操作，不管其他进程的状态。当有消息返回时系统会通知进程进行处理，这样可以提高执行的效率

1. get请求：从服务器获取数据，需要服务器返回响应

2. post请求: 给服务器提交数据

- HTML

1. 设置类标签, 不显示
2. 显示类标签, 如
ui li form input 录入交互, 展示img或table, 效果
3. `<a>` `<button>`

- CSS重叠样式表

1. 功能: 定位 box模型 显示样式 动画 响应
2. 使用方法: style属性, style标签, link标签(链接外部css,javascript等文件)等
3. 选择器:
 1. `a{ }` //所有 `<a>` 标签
 2. `#myid{ }` // `<a id = "myid">`
 3. `.cls{ }` // `<a class = "cls cls1 cls2">` 重叠定义
 4. `.cls div{ }` //衍生, `.cls` 里的div标签
 5. `.cls>div{ }` //衍生, `.cls` 里的直属子div标签
 6. `.cls.aa{ }` // `<a class = "cls aa">`
 7. `.cls, .aa{ }` // `<a class = "cls">` 或 `<a class = "aa">`
 8. `select option:selected{ }` // `<select>` 里选中的 `<option>` 标签, 伪类选择

- JavaScript用户交互

1. BOM -> 浏览器对象模板, location.href="" history.back()
2. DOM -> 文档对象模板, 每个标签都可作为对象来进行操控
3. ajax -> 异步通讯, store
4. 面向对象开发
5. 事件处理

- JQuery组件

1. `$("<选择器>")` 与 js标签对象的关系 -> `var e1 = $("选择器")[0];`
val html text attr prop css 均用于修改标签内容及样式
2. append() remove() 动态添加/移除
3. 事件定义

```
//way1.
$("body").on("click", "button", function (){    //为body中的button组件添加
click事件, 预定义形式
    var a = $("<a>").attr("href", "#").text("巴拉巴拉");    //自动分行
    $("body").append(a);
});

//way2.
$("button").on("click", function(){})    //常规形式
```

1. 使用动态方法创建的标签或组件只能通过预定义来进行修饰其各项属性

2. 冒泡处理

一个内部组件事件的触发实际上会一层层向外层组件发送信号，由内向外的触发，即便事件只是定义在内部组件上

4. 创建标签

```
var a = $("<a>").attr("href", "#").text("巴拉巴拉");
$("html脚本")
```

5. next prev parent 检索方法

6. ajax get post 异步通讯

7. \$("选择器").add(js标签)

例如：

```
layui.use(["layer", "jquery", "laydate"], function () { //jquery选择器格式
    var layer = layui.layer;
    var laydate = layui.laydate;
    var $ = layui.jquery;
    $(".mydate").css("color", "red");
});
```

8. jquery动画

- 前后端通讯请求发送方式： -> GET/POST

1. 地址重定向，如：<a> location.href=" ", open("地址") 与浏览器地址栏相关

-> GET 地址上全部内容 -> http://ip|域名|主机名:端口/应用名/地址映射?参数名=参数值&参数名=参数值

GET安全指数并不高，在要求安全指数高的情况下尽量不用GET，大多数会选择POST

2. form表单 GET POST

3. ajax请求 GET POST -> 异步请求

在浏览器内调试时，可以在 审查元素 - 网络 - Fetch/XHR 中查看提交的GET请求

- JS面向对象

```
function objName(构造参数){
    var a = 1; //临时变量，只能在对象内使用
    this.b = 2; //实例变量，在对象内用this.variable来使用，对象外可以直接调用
    function fun(){}; //内部函数，只能在对象内部调用
    this.init = function(){ //实例函数
        this.c = 0;
    }
}
```

1. 关于layui框架的使用

```
//初始化日期选择器组件，即创建一个日期选择器类，若要使用还需要在<body>中实例化对象并指定为mydate类
layui.use(["layer", "jquery", "laydate"], function () { //常规格式
    var layer = layui.layer;
    var laydate = layui.laydate;
    laydate.render({
        elem: '.mydate',
        type: 'date',
        format: 'yyyy年MM月dd日'
    });
});
```

- 前端框架layui
 1. lay-filter //事件触发过滤，可用于定义事件，以指向具体实现代码
 2. lay-event //事件源过滤，用于主部件触发时指向内部子部件
- 状态列显示
 - 举例

```
{field:'statesname', title:'状态', width:100}
```

```
//written in entity/User_project.java
public static String[] STATUS = {"在职", "离职"};
public String getStatus(){
    return STATUS[status];
}
```

- 增改

```
<select name="status"></select>
ajax_array(fm, $("[name = status]"), "getArray?name=STATUS", 0);
```

- 外键列显示

1. 改代码

```
{field:'type_name', title:'类型', width:100}
```

```
//in entity/User_project.java
private String type_name;
```

2. 数据库中新增视图，并且在对应mapper文件中加入@TableName("v_user")

1. 创建映射表

2. 创建映射关系，即视图

一般会将被映射表中的 `<type_id>` 映射到映射表中的 `id`

然后将映射表中表示对应名称的值，如 `name` 等，赋一个别名，以便在包含了被映射表对应pojo对象的index.html中使用，也就是可以直接使用别名(更清晰地明白使用属性的含义)

3. 在被映射的pojo对象中使用@TableName("<视图名>")添加视图

4. 在index.html中将 `<type_id>` 替换为视图中映射的别名即可

3. 实现伪删除 isdel 列

- 增改

```
<select name="type_id"></select>
ajax_list(fm, $("[name = type_id]"), "/UserType/getAll");
```

- 一般在controller中调用service, 在serviceImpl中调用mapper
- <HttpServletResponse类型对象>.setContentType("<MIME类型>") //让浏览器将接收到的信息视为MIME中的某一类型
- 密码加密
若用MD5等加密方式对密码进行了加密, 则其存储在数据库中的数据是加密后的数据, 直观表现大概是一串字符
更改加密方式后, 切记将数据库表中密码字段值也修改为对应加密后的密码字段值

工程项目设计

- 类似于商品数量一类的数据, 最好不要在表中用字段来记录, 而是用统计表来记录变化情况, 类似于日志
- 在User表中可以加入isdel字段值, 实现伪删除, 并重新实现删除方法(记得更改表中字段后, 相关视图和pojo对象也要进行增删)
- 项目中要减少代码的嵌套性, 尽量使用排除法进行条件筛选判断等操作
- controller中一般只放网页层面交互, 业务相关操作都放在service包中
- 修改密码
 1. 在 user_serviceImpl 中创建repass方法, 接收三个String参数及HttpSession参数

```
@RequestMapping
public ResultData repass(String newPass, String oldPass, String
confirmPass, HttpSession session){

}
```

- 项目要素
 1. 客户需求 -> 说服客户使用自研发的产品, 而不是同行业的竞品
 1. 软件的服务客户需求, 例如, 超市管理系统, 则要考虑购物客户的需求
 2. 使用人员的需求, 买项目的人
 3. 管理人员的需求, 应用公司管理层人员的需求
 4. 目标客户
 5. 客户痛点
 2. 产品使用需求
 1. 用户体验
 2. 业务闭环
能否让a业务和b业务连续周转起来
 3. (次要)美化
 3. 管理需求

1. 以管理目标为主(管理数据) -> 能否快速精准地了解公司的运行状况，或者及时获取各方面信息、提示预警
2. 可发展性 -> 提高收益，不进则退

手机售后管理项目

3个业务表

1. 流程组件

1. 员工
2. 配件
3. 库存
4. 工作台
5. 产品

2. 流程

1. 修复业务(生命周期) —— 主线

产品 -> 员工 -> 工单 -> 配件 -> 修复人员 -> 完工结算

2. 产品

产品档案 -> 维修记录

3. 配件周期

进货 -> 存储 -> 出库

4. 工单

工单创建 -> 维修方式 -> 维修人员 -> 客户评价

3. 数据库表

1. 员工类型

2. 员工

1. 加入 权限(power):int

2. User表

3. 品类表

1. name

4. 型号表

1. name

2. 品类id

3. 型号信息

4. 配置表ids

5. 配件表

1. name

2. 配置类型id

3. 默认保修期

4. 规格

5. 可支持配件信息

6. 数量

7. 建议销售单价

6. 配件类型表

1. name

2. 类型

1. 内存

2. 显示屏

7. 产品档案

1. 档案手机编号

2. 型号id

3. name

4. tel

5. pass

6. 创建人

7. 创建日期

8. 购买日期

9. 状态

1. 未接单

2. 进行中

3. 未结算

4. 已结算

10. 保修期

11. 发票号

12. 备注

8. 工单

1. 档案id

2. 开单时间

3. 接手人

4. 联系电话

5. 接手时间

6. 开始时间

7. 结束时间

8. 之前状态信息

9. 之后状态信息

10. 工单状态

1. 未进行

2. 进行中

3. 已完结

- 11. 备注
 - 12. 工单类型
 - 13. 等待配件
 - 14. 发回厂家
 - 15. 安装维修
 - 16. 维修费
 - 9. 进出库记录
 - 1. 日期
 - 2. 货单号
 - 3. 工单号id
 - 出库针对哪个工单
 - 4. 目标人
 - 5. 配件id
 - 6. 实际保修期
 - 7. 数量
 - 8. 成本单价
 - 9. 单价
 - 10. 金额
 - 11. status
 - 12. type处理类型
 - 1. 进
 - 2. 出
 - 3. 损益
 - 13. 备注
 - 14. 审验人
 - 15. 审验日期
 - 16. 收费状态
 - 10. 维修收款单
 - 1. date
 - 2. 档案id
 - 3. amount
 - 4. paytype
 - 5. 实收金额
 - 6. 收款人
 - 4. 客户痛点
 - 1. 信用 品牌专业性
-

异常调试

- 网络状态号：可在 [浏览器审查元素 - 网络 - 状态号 查看](#)
 1. 200 -> 正常
 2. 300 302 304 -> 300系列状态号主要是请求被取消
 1. 浏览器缓存问题
 2. 被服务器重定向
 3. 400 403 404 406 -> 400系列主要是请求被拒绝
 1. 404 -> 服务器无法为本次请求服务
 2. 400 -> 浏览器端返回的数据类型与服务器需要的类型不匹配，例如：浏览器 (bir="admin") 服务器(Date bir)
 3. 目标对象不存在
 4. 拒绝进入目标代码
 4. 500 -> 目标代码异常 服务器端代码异常
- 1. 前端
 1. js代码 -> 浏览器控制台
 2. 404错误 -> 浏览器控制台
- 2. 后端
 1. 在终端看报错，找异常名称和第一处报错
- 3. 若某一项数据在网页中未正常显示，可按以下步骤进行错误排查
 1. 在前端html文件中找到展示该数据的对应标签，检查其使用的变量名等属性与数据库、pojo对象中的变量名是否一致
 2. 若已经确认前端相关变量已经修改无误，但网页中仍未正常显示数据，则可以尝试让浏览器重新加载框架，或刷新浏览器缓存